The theorem is on all three overlays, 1, 2, 3. Its illustration is on overlays 2 and 3, leaving room for the proof that appears only on overlay 3, This is an example of "out of sequence overlays". We code this in `babybeamer6` (in the `samples` folder).

Since declarations, proofs, and the `\includegraphics` command may all have overlay specifications, this seems easy to accomplish.

```
%babybeamer6 presentation, first try
\documentclass{beamer}
\begin{document}

\begin{frame}
\frametitle{Overlaying declarations and graphics}
\begin{theorem}<1->
Every finite distributive lattice can be embedded
in a boolean lattice.
\end{theorem}
\begin{proof}<3->
Use join-irreducible elements.
\end{proof}
\includegraphics<2,->{cube}
\end{frame}
\end{document}
```

This does not work too well. On overlay 1 the theorem appears in the middle and then it jumps up to make room for the illustration. This is the same problem we encountered in the `babybeamer2` presentation in Section 12.2.1 and the solution is also the same, the use of the `\onslide` command. Replace the line

```
\includegraphics<2,->{cube}
```

with

```
\onslide<2->{\includegraphics{cube}}
```

12.2.6 Blocks and overlays

You can think of a theorem in `beamer` as the contents of the `theorem` environment with a heading and, optionally, with an overlay specification, and with most themes—see Section 12.5—colorful visual highlighting, see Figures 12.3 and 12.4.

`beamer` provides the `block` environment that works the same way except that you name the block. The (partial) syntax of the `block` environment is

```
\begin{block}<overlay spec>{title}
source
\end{block}
```

Blocks are shaped as theorems. If there is no title, you still need the braces. The overlay specification is optional.

As an example, save babybeamer6.tex as babybeamer6block.tex in the work folder (also in the samples folder along with the PDF file) and replace the theorem environment with

```
\begin{block}<1->{Theorem}
  Every finite distributive lattice can be embedded
in a boolean lattice.
\end{block}
```

If you want a block of LaTeX code with an overlay specification but with no title and no visual highlighting, use one of the commands, \onslide and \only.

12.2.7 Links

A presentation is a PDF file, so it is not surprising that you can set links of various types in a beamer presentation. Just as the hyperref package helps us with hyperlinks in a PDF file (see Section 11.2), the beamer package allows us to conveniently set links in a presentation.

Some links are automatically provided. If you look closer at Figures 12.1 and 12.2, you see that the section titles are shown in the *sidebar*. In fact, the sidebar is a *navigation bar*. First, it shows which section you are in. Second, clicking on a section title takes you to that section.

Creating a link is a two-step process.

1. Name the place you want to link to.
2. Create a button with the property that clicking on it jumps you to the designated place.

To illustrate this process, we modify the presentation babybeamer4. Open the file babybeamer4.tex and save it as babybeamer7.tex in the work folder (the edited version is in the samples folder along with the PDF file).

1. Name the frame you want to link to by adding a label to the \begin{frame} line. In babybeamer7, add a label to the frame fourguests:

   ```
   \begin{frame}[label=fourguests]
   ```

   Labels of frames are also useful for selective typesetting of your presentation, see Section 12.6.

2. Add the following line to babybeamer7:

   ```
   \hyperlink{fourguests<3>}%
             {\beamergotobutton{Jump to third guest}}
   ```

This creates a link to the third overlay of the frame named `fourguests`, and creates a button, with the text `Jump to third guest`. Clicking on this button will jump to the third overlay of the frame `fourguests`.

3. To add variety to linking, include a new first frame:

```
\begin{frame}
\frametitle{First frame with a button}
Button example

Jumping to an overlay of a different frame
\bigskip

\hyperlink{fourguests<3>}%
{\beamergotobutton{Jump to third guest}}
\end{frame}
```

which has a button for jumping to the third overlay of the `fourguests` frame.

4. We also add a new third frame.

```
\begin{frame}
\frametitle{Third frame with a button}
Button example

Jumping to another frame
\bigskip

\hyperlink{fourguests}%
{\beamergotobutton{Jump to guest list}}
\end{frame}
```

with a button, with the text `Jump to guest list`. Clicking on this button will jump to the second frame, overlay not specified (defaults to 1).

5. Add a fourth frame,

```
\begin{frame}
\frametitle{Hidden link}
\hyperlink{fourguests}{Jumping to the guest list}
\end{frame}
```

introducing another version of the `\hyperlink` command:

```
\hyperlink{fourguests}{Jumping to the guest list}
```

which typesets the second argument as regular text, making it an *invisible link*. However, you may notice that the cursor changes when it hovers over the link. For in-

stance, you may want to link the use of a concept to its earlier definition, where you also need a button for the return jump.

Here is babybeamer7:

```
%babybeamer7 presentation
\documentclass{beamer}
\begin{document}

\begin{frame}
\frametitle{First frame with a button}

Button example

Jumping to an overlay of a different frame
\bigskip

\hyperlink{fourguests<3>}%
{\beamergotobutton{Jump to third guest}}
\end{frame}

\begin{frame}[label=fourguests]
\frametitle{Overlaying lists}

We introduce our guests:
\begin{itemize}
\item<1-> R. Padmanabhan
\item<2-> Brian Davey
\item<3-> Harry Lakser
\item<4-> Dick Koch
\end{itemize}

\hyperlink{fourguests<3>}%
{\beamergotobutton{Jump to third guest}}
\end{frame}

\begin{frame}
\frametitle{Third frame with a button}
Button example

Jumping to another frame
\bigskip
```

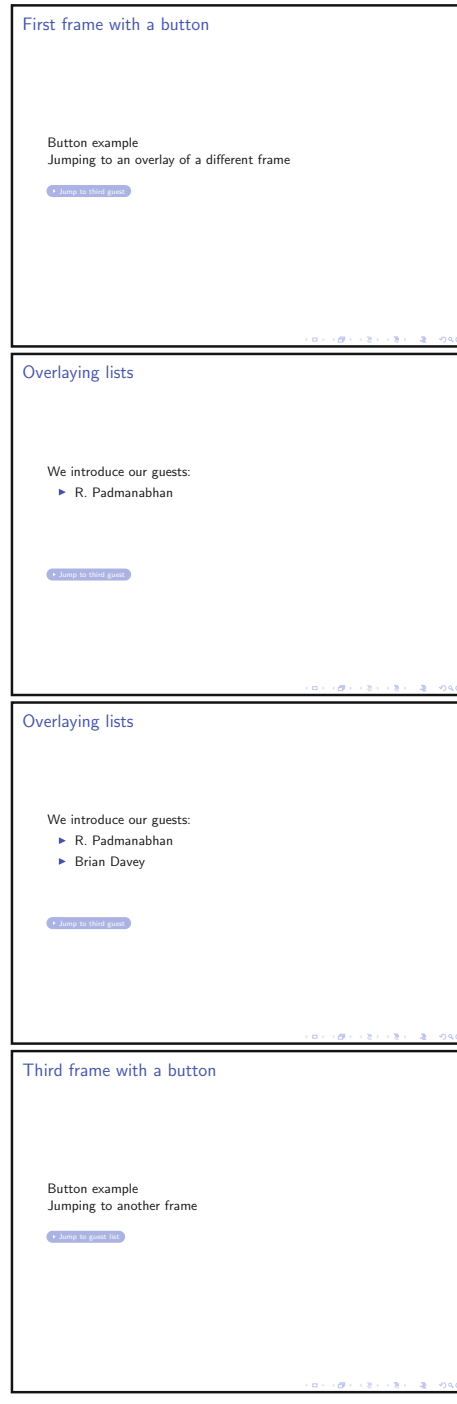


Figure 12.9: `babybeamer7` presentation.

```
\hyperlink{fourguests}%
{\beamergotobutton{Jump to guest list}}
\end{frame}

\begin{frame}
\frametitle{Hidden link}

\hyperlink{fourguests}{Jumping to the guest list}
\end{frame}
\end{document}
```

Figure 12.9 shows all these buttons. We do not show overlays 3 and 4 of frame 2 and frame 4, where the button is invisible.

12.2.8 Columns

Often, it is useful to put the display into columns. A simple illustration is given in babybeamer8:

```
%babybeamer8 presentation
\documentclass{beamer}
\begin{document}

\begin{frame}
\frametitle{Columns, top alignment}

\begin{columns}[t]
\begin{column}{2in}
Is it true that there is no new result
on the Congruence Lattice Characterization Problem?
\end{column}
\begin{column}{2in}
F. Wehrung found a distributive algebraic lattice that
cannot be represented as the congruence lattice
of a lattice.
\end{column}
\end{columns}
\end{frame}
\end{document}
```

The environment is `columns`. It has an optional argument for alignment, `t` for top, `c` for center, and `b` for bottom. The columns, usually two, are both in the `column` environment; the width of the column is in the argument; it can be given as a measurement—`2in` in the example—or relative to the width of the whole frame as `0.4\textwidth`.

Figure 12.10 shows the `babybeamer8` presentation.

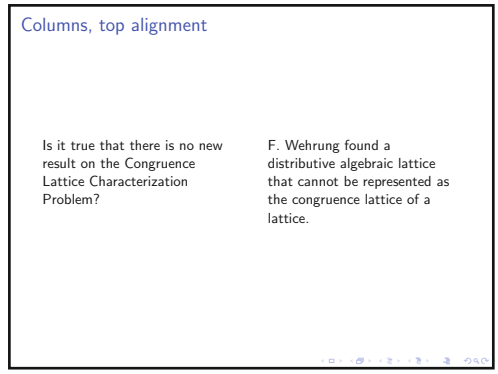


Figure 12.10: `babybeamer8` presentation.

12.2.9 Coloring

LaTeX's job is to produce articles and books that contain text, math formulas, and graphics. Such publications—with the exception of textbooks—cannot afford color printing. Presentations are different. If you prepare a color presentation, it will project in color.

Nevertheless, the color commands are of limited use even for presentations. You probably use the color scheme of the chosen theme (see Section 12.5), and have limited opportunity to color things yourself. If you do, be very careful, too much color distracts from the presentation but judicious use of color—say, for highlighting a word or phrase—may be very effective.

`beamer` uses the sophisticated `xcolor` package of Uwe Kern. It colors by specifying the color model: `rgb` (red, green, blue), or `cmyk` (cyan, magenta, yellow, black), or `gray` (black and white)—there are many more models to choose from—and how much of each color you want to mix.

So `\color[rgb]{0,1,0}` paints everything—within its scope—green. You can color some text green with the command

```
\textcolor[rgb]{0,1,0}{This text is green.}
```

There are seventeen predefined colors: red, green, blue, cyan, magenta, yellow, orange, violet, purple, brown, pink, olive, black, darkgray, gray, lightgray, and white. With the proper options, there are hundreds more. So the previous command could also be given as

```
\textcolor{green}{This text is green.}
```

or as

```
{\color{green}This text is green.}
```

To pretty things up, you can use `\colorbox{green}{Green box}`, which puts the argument in a green box and `\fcolorbox{red}{green}{Green box}`, which also adds a red frame.

`xcolor` is automatically loaded by `beamer`. To make sure that `xcolor` is loaded with the options desired, you have to include these options in the preamble in the

```
\documentclass{beamer}
```

line. For instance, to have the `dvipsnam` option for `xcolor`, invoke `beamer` with

```
\documentclass[xcolor=dvipsnam]{beamer}
```

You can also mix predefined colors:

```
{\color{green!40!yellow} This text is of what color?}
```

which sets the text 40% green and 60% yellow.

There are commands for defining colors and color sets, as well as for coloring the background, frames, and hyperlinks (see Uwe Kern, *Extending LaTeX's color facilities: the xcolor package* [46]).

Here is a simple illustration:

```
%babybeamer9 presentation
\documentclass{beamer}
\begin{document}

\setbeamercolor{normal text}{bg=yellow!15}
\begin{frame}
\frametitle{Colors}

\begin{columns}[t]
  \begin{column}{2in}
{\color{red}Is it true that there is no new result
on the Congruence Lattice Characterization Problem?}
  \end{column}
    \begin{column}{2in}
{\color{green}F. Wehrung found a distributive
algebraic lattice that cannot be represented
as the congruence lattice of a lattice.}
  \end{column}
\end{columns}
\end{frame}
\setbeamercolor{normal text}{bg=green!15}
```